Very Important Notice

Resources on the compute cluster are **limited**. Importantly, there is no per-user constraint, but rather a **hard limit on the entire group**. Therefore, please make sure you don't use more than 75 gpu-hours of computation! Otherwise, other people may not be able to complete the assessment!

If you think you need more resources, please contact one of the coordinators (Jakub Krajewski, Michał Krutul) and we will try to find a solution.

You can print gpu-hours used by logging into the cluster and running the following command:

```
echo "GPU hours used: $(hpc-jobs-history -A plgpkattenion-gpu-a100 -d 250 | awk '$11 ~
/^[0-9.]*$/ {sum += $11} END {print sum}')"
```

Organizational

- **Compute cluster:** In the description below, we assume you will use the Athena cluster (access is provided for all students). However, the setup (account creation/acceptance) takes some time. Therefore, in case of any issues, you are free to use any other cluster which allows for multi-gpu training.
    - If you use Entropy or other cluster with older gpus: use float16 as the Mixed Precision dtype. Also, for point 7, you can calculate the compute-optimal number of tokens and report that value, but you can run the actual training for fewer tokens so the experiment finishes in a reasonable time. A full run for one experiment from the grid in point 7 should take at most about 1 hour. Example wording for the report: "The compute-optimal number of tokens is <>. Since I am using a cluster with older GPUs, I trained the models for <> tokens to keep the training time reasonable."

- **Deadline:** The deadline for completing the project is **Saturday, January 31st, End of Day (23:59)**.
- Late submissions policy: Your points will be rescaled for every 12 hours of delay. Let d denote your delay in hours and p your points for solving the assignment. Then:
    - you get 0.9*p score if $0 < d < 12$
    - you get 0.8*p score if $12 \leq d < 24$
    - you get 0.7*p score if $24 \leq d < 36$
    - you get 0.6*p score if $36 \leq d < 48$
    - you get 0.4*p score if $48 \leq d < 60$
    - you get 0.2*p score if $60 \leq d < 72$
    - you get 0 score otherwise
- **Deliverable:** Described at the bottom of the document.

- In the report, **please specify how many gpu-hours you have used to complete the project.** You can find the appropriate command above.

Please plan your work, as your job submissions may spend some time waiting in the queue. Therefore it is recommended to start working on the project as soon as possible.

Task

The starting point of this project is a basic implementation of Transformer pretraining, available here:
https://drive.google.com/drive/folders/1qFiWfVJCjMV10Y5KxBtpwnUNo06b6xc6?usp=sharing
Currently, it allows to pretrain the model on C4 dataset in full (float32) precision, on 1 GPU. Your goal in this project will be to change and optimize the codebase, to allow for efficient training of large models on multiple devices.

Setup

- Log in to the cluster
- Download the code to the storage directory: `/net/tscratch/people/<username>`
- Create a Python venv and activate it
- Install requirements in the venv (listed in the requirements.txt file)
- Request 1 gpu using the `srun` command from the Athena setup instructions doc (or corresponding command on another cluster you are using)
- Once your gpu resources are allocated, verify the training is working by:
  - activating venv
  - running `python main.py`

Steps to Implement

Please save all `sbatch` scripts used in the points below, and add them to the zipped code you will send. Please make sure all scripts are run just by `sbatch <script_name>.sub` - to make sure all used arguments are visible in the script.
Below are the steps you need to implement, with corresponding points for completing each part. For each step, verify the training is working by running the training for 5 minutes.

1. Create an `sbatch` script to be able to easily submit jobs. Save the script as `submit_begin.sub`. **1 point**
2. Add `ArgumentParser` to your script, to be able to change hyperparameters, like `n_layers, dmodel, n_heads, batch_size, n_training_steps` from the command line when running the script rather than by making modifications in the main.py file. **1 point**
3. Add logging of model train loss to an online monitoring platform of your choice (we highly prefer Neptune or wandb). This will help you easily track experiments. Make sure arguments from `ArgumentParser` are logged to the service as well (for example, as tags). **1 point**

4. Implement the necessary distributed training setup **12 points**:
    a. Modify the `sbatch` script to run your job using `torchrun`. Save the new `sbatch` script as `submit_torchrun.sub`
    b. Modify the code to use FSDP. Make sure your training will use Mixed Precision (you can set both `param_dtype` and `reduce_dtype` to `bfloat16` or `float16`).
    c. Run your training on 2 gpus within one node (arguments `--nodes=1 --gpus=2` in the batch script). Save the `sbatch` script as `submit_2gpu.sub`.
    d. Modify the data loading process to make sure each device loads unique batches of data.
5. Add Warmup-Stable-Decay (WSD) learning rate schedule with warmup (this simple lr scaling strategy is described in https://arxiv.org/abs/2404.06395). The warmup should take 1% of the training steps, and the decay should take 10% training steps. Verify the lr is set properly by logging it to experiment tracker . **4 points**
6. Implement and change the activation function in the FF layer to SwiGLU (https://arxiv.org/pdf/2002.05202). **5 points**
7. Calculate optimal number of tokens (hence, training steps) by extrapolating optimal compute value from Chinchilla Scaling Laws. For simplicity, you can count the number of model parameters N and assume the number of training tokens D=20N is compute optimal. Run small grid search with the following hyperparameters:
    - number of steps: determined by extrapolation
    - learning rates: [1e-2, 1e-3, 1e-4] (each experiment with different learning rate)
    - num_layers: 4
    - num of attention heads: 4
    - dmodel: 256
    - sequence_length: 256
    - batch_size: 256
    - dropout: 0.0

    on 2 gpus, where each gpu is on a different node (`--nodes 2 --gpus 2` in `sbatch` script). Use **slurm**
    job array. Save the `sbatch` script as `sbatch_grid_search.sub`. **6 points**

Deliverable

The zipped folder should be sent using this form: https://forms.gle/Lu2VQBS77ofD15cb6

The folder should contain:

- source code - main.py (and possibly other source files)
- all properly named `sbatch` scripts
- PDF report including:
    - very concise explanation of how you implemented features

- plot of learning rate from experiment tracker showing correctly implemented learning rate scheduler
- brief explanation of your calculation of compute optimal steps
- plots from experiment tracker service proving multigpu training (for example similar to the ones below):

- plots comparing losses from experiments with different learning rates (show loss curves for different learning rates on one plot)